Name - Pulkit
Artificial intelligence InternsElite
Batch – June 2026

AI-Driven Placement & Mock Interview Evaluation Hub

Abstract

Job seekers preparing for technical placements typically face two disconnected steps: checking whether their resume matches a target job description, and rehearsing role-specific interview questions with useful feedback. This project brings both into a single Streamlit application powered by a large language model (Meta's Llama 3.3 70B, served through Groq). The first module scores an uploaded resume against a pasted job description and highlights missing keywords and structural fixes; the second module generates a role-specific interview question, accepts a typed answer, and returns a graded evaluation with strengths, flaws, and a model reference answer. Both modules use LangChain to structure prompts and enforce a strict JSON response schema so the results can be rendered directly as UI widgets.

Introduction

Applicant Tracking Systems (ATS) are the first filter almost every resume passes through before a human ever reads it, so a resume that is well-written but poorly keyword-aligned with the job description can be rejected automatically. At the same time, candidates preparing for technical interviews often lack a low-stakes way to practice role-specific questions and get structured feedback on their answers rather than a generic pass/fail. Large language models are now capable of both tasks — comparing unstructured text against a rubric, and generating and grading domain-specific questions — provided their output is constrained to a predictable, parseable format.

This project, the AI-Driven Placement & Mock Interview Evaluation Hub, combines these two needs into one placement-preparation tool: an ATS Screening pillar that scores resume-to-job-description alignment, and an Interactive Mock Interview pillar that runs a question-answer-grade loop for a role the candidate specifies.

Problem Statement

Build a single interactive application that (1) accepts a candidate's resume (PDF) and a target job description, and returns a quantitative match score along with missing keywords and resume-restructuring advice, and (2) accepts a target role, generates a relevant technical interview question, accepts the candidate's typed answer, and returns a structured grade with strengths, flaws, and a reference answer — all without needing a curated question bank, by delegating question generation and evaluation to an LLM constrained to a fixed JSON schema.

System Architecture

The application is a single Streamlit script with sidebar-driven navigation between the two pillars. Both pillars follow the same underlying pattern: collect input, extract or hold it in session state, build a LangChain prompt

around it, send it to ChatGroq, and parse the model's JSON reply into the metrics and text blocks shown below.

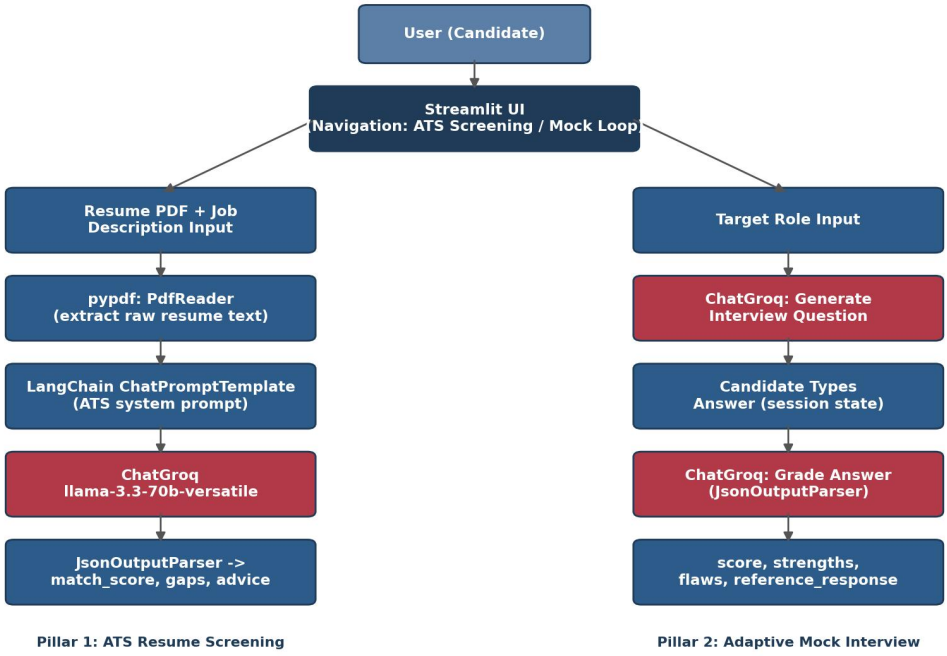

Figure 1: High-level data flow for the ATS Screening pillar (left) and the Adaptive Mock Interview pillar (right).

Methodology & Tech Stack

Component	Choice
Frontend / app framework	Streamlit (sidebar radio navigation, session_state)
LLM provider	Groq API (fast inference)
Model	llama-3.3-70b-versatile
Orchestration	LangChain (ChatPromptTemplate, chain composition)
Output parsing	LangChain JsonOutputParser (strict JSON schema)
Resume parsing	pypdf (PdfReader, per-page text extraction)

Module 1 — ATS Resume Screening

The user uploads a PDF resume and pastes a job description into two side-by-side inputs. pypdf.PdfReader walks every page of the uploaded file and concatenates the extracted text. That raw resume text and the job description are substituted into a system/human prompt pair that instructs the model to act as a technical recruiter and return exactly three JSON keys: match_score (0-100), missing_tech_keywords, and resume_reconstruction_advice. The chain (prompt | llm | JsonOutputParser) returns a Python dict that is rendered directly as a percentage metric plus two text blocks.

Module 2 — Adaptive Mock Interview

The user types a target sub-domain (e.g. "Backend Developer"). A first prompt asks the model to output a single challenging interview question for that role with no conversational filler, which is stored in st.session_state.interview_question so it survives the next rerun. The candidate then types an answer in a text area; a second, stricter prompt (temperature 0.2, to keep grading consistent) asks the model to return score, technical_strengths, logical_flaws, and reference_response as JSON, which are rendered as a score metric, two feedback blocks, and a code-formatted model answer.

Illustrative walkthrough

The Groq API key used by this app is a personal credential and was not called to produce this report. The example below illustrates the expected shape of the output for a representative input, based directly on the prompts defined in the code — it is not a live model response.

Field (ATS Screening)	Illustrative value
match_score	72
missing_tech_keywords	Docker, CI/CD, AWS, GraphQL
resume_reconstruction_advice	Move tech stack to a top summary line; quantify project impact with metrics
Field (Mock Interview)	**Illustrative value**
Generated question	Design a rate limiter for a public API — compare token-bucket vs. sliding-window and how you'd scale it across servers
score	68
logical_flaws	Did not address distributed/multi-server synchronization of rate-limit state

Limitations & Recommended Fixes

A close read of the current implementation surfaces a few issues worth addressing before this is shared or deployed beyond local testing:

- Hardcoded API key: the Groq key is set directly via os.environ["GROQ_API_KEY"] = "..." in the script. Any hardcoded key is exposed the moment the code is shared, committed to version control, or posted anywhere. It should be moved to an environment variable set outside the code, or to Streamlit's st.secrets, and the currently hardcoded key should be revoked and replaced.
- Mock Interview button/else logic: "if st.button(...): ... else: st.error(...)" shows the error on every rerun where the button was not just clicked — including the very first page load — because st.button() is False by default, not just when validation fails. The error should instead be conditioned on an empty role_input, independent of the button's momentary state.
- Unused role in grading: st.session_state.target_role is initialized to an empty string and never assigned when the user types a role (role_input is a separate local variable), so the grading prompt's {role} placeholder is sent as blank text instead of the actual target role. Assigning st.session_state.target_role = role_input when the question is generated would fix this.

- No retry / error handling around the LLM calls: a malformed JSON response, a rate-limited request, or a network error from Groq will raise an uncaught exception and crash the current interaction rather than showing a friendly retry message.

Conclusion & Future Scope

The AI-Driven Placement & Mock Interview Evaluation Hub demonstrates that a small set of well-constrained LLM prompts — one per task, each locked to a strict JSON schema — is enough to build a genuinely useful two-in-one placement-prep tool without any custom ML model training or a curated question bank. The architecture cleanly separates the two pillars while reusing the same prompt -> LLM -> JSON-parse pattern throughout, which keeps the codebase small and easy to extend.

Beyond fixing the issues above, natural next steps include persisting past mock-interview attempts so a candidate can track score trends over time, extending the ATS module to compare a resume against multiple job descriptions at once, adding support for DOCX resumes alongside PDF, and swapping the single-question interview loop for a multi-question session with a final consolidated report.

Reference

LangChain Documentation, https://python.langchain.com

Groq API Documentation, https://console.groq.com/docs

pypdf Documentation, https://pypdf.readthedocs.io

Streamlit Documentation, https://docs.streamlit.io